

Prediction Markets Dashboard

Cross-Exchange Analytics for [Polymarket](#) × [Kalshi](#)

Nikhil Pothuganti · July 20, 2026

Next.js 15 · TypeScript · 6,372 LOC · 3 routes · 11 API endpoints

1 The Problem

Prediction markets are fragmented. The same question — who wins the 2028 US election, who lifts the 2026 World Cup — trades on both [Polymarket](#) and [Kalshi](#), at different prices, under different resolution rules.

A gap between the two venues looks like free money. It usually isn't.

This dashboard pulls live order books and price history from both venues, works out which markets are the same question, then tests every gap three ways:

- **Is it fillable?** Walk both books. Don't quote the touch.
- **Does it survive fees and time?** [Kalshi](#)'s fee is non-linear, and capital is locked until the market resolves.
- **Do the two markets actually pay the same?** Often they don't.

Thesis. A visible price gap is by default **not** an edge. A spread is shown as tradeable only when the match is `VERIFIED`, the depth- and fee-aware edge is positive, and the divergence assessment sits beside it.

Result. On the 2028 President market, an apparent 8-point gap is really about **0.8%/year**. A T-bill beats it. And the venues don't settle on the same event — [Kalshi](#) pays on who is *inaugurated*, [Polymarket](#) on the *AP/Fox/NBC race call*. It was never a lock.

That thesis drives everything: why the matcher rejects rather than guesses, why the engine walks the book, why returns are annualised, why the divergence layer exists.

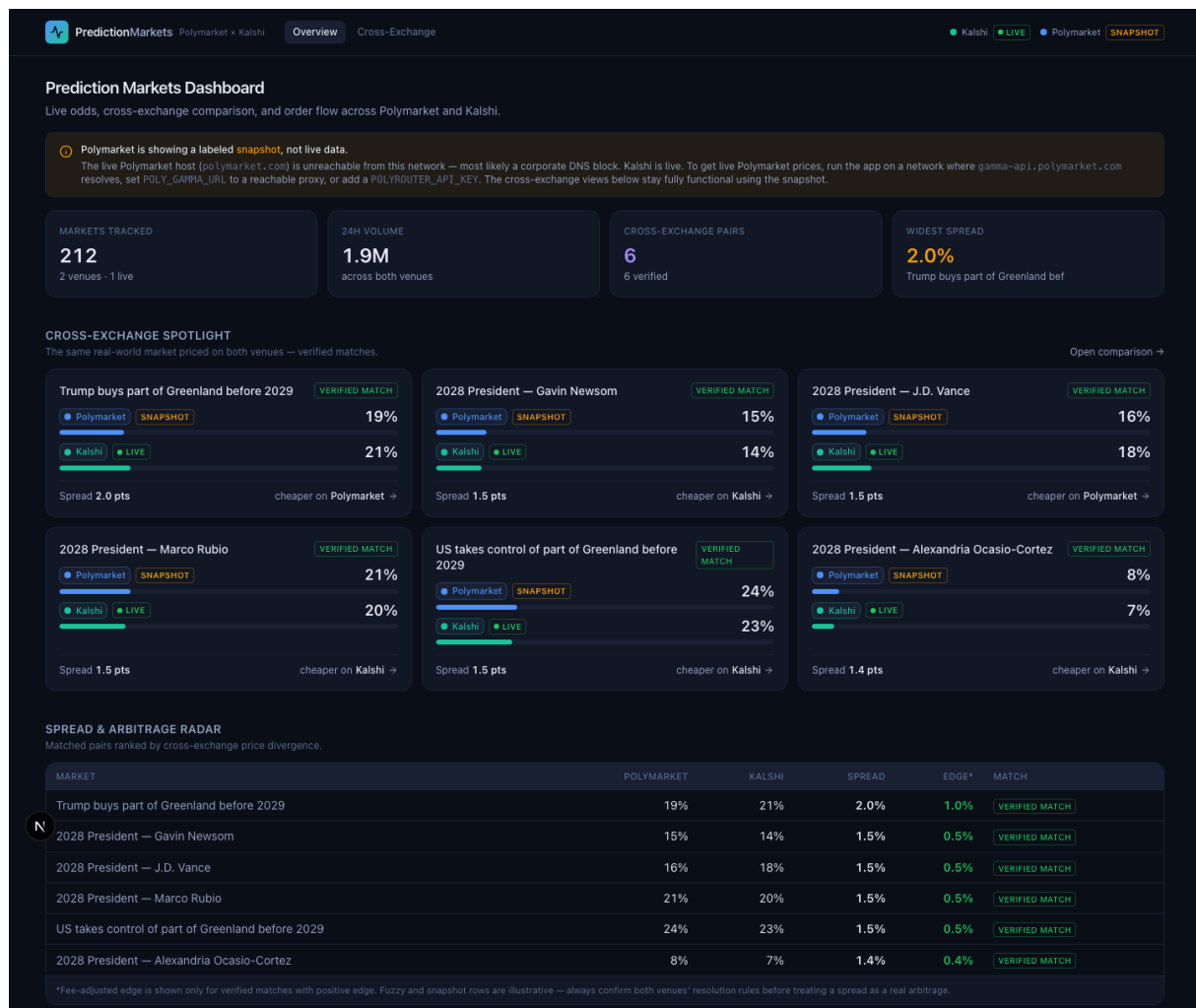


Figure 1. Overview. Badges top-right show **Kalshi** LIVE and **Polymarket** in labeled SNAPSHOT mode (§6). Below: corpus stats, verified pairs, and the arbitrage radar ranked by divergence.

2 Architecture

One Next.js 15 App Router deployment. No separate backend — the server tier is route handlers under `app/api/`, and they are the only thing that talks to a venue.

That isn't stylistic. **Kalshi** returns 403 to any request carrying an Origin header, so a browser fetch cannot work. Proxying server-side also puts normalisation, caching and degradation in one place.

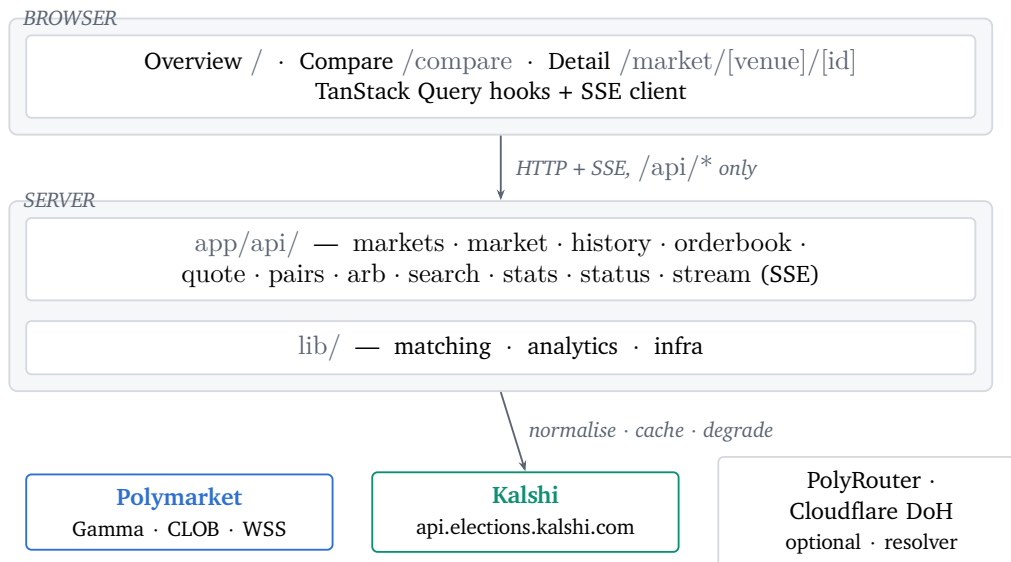


Figure 2. Request topology. Caches are prewarmed at boot, so the first request doesn’t pay the ~17 s cold build. Most of the code is in `lib/` (3,297 LOC) rather than `components/` (2,342) — a data pipeline with a UI, not the reverse.

Feature	Summary
Cross-exchange comparison	Same question on both venues: dual-line history, live spread, both books, resolution rules side by side
Depth-aware arbitrage	Walks both books until the edge collapses; reports fillable size, net profit, annualised ROI
Resolution divergence	Flags matched pairs that would settle differently
Three-tier matching	Entity alignment, curated seed, IDF-weighted fuzzy fallback
Real-time order book	Polymarket over WebSocket → SSE; Kalshi polls at 2 s
Arbitrage radar	Matched pairs ranked by divergence, filterable by category
Market browser	Both venues, server-side sort and filter, debounced search, infinite scroll
DNS-over-HTTPS	Live Polymarket data on networks that DNS-block it. No VPN
Per-venue degradation	LIVE/SNAPSHOT/OFFLINE, labeled everywhere
Performance	Five SWR cache layers, boot prewarm, incremental chart updates

Table 1. Feature summary. The three that distinguish this from a spread scanner are covered in §3–5.

3 Matching

The hardest part. There is no shared identifier between venues, so the join is inferred from text — and a false positive is worse than a miss, because it invents a spread between two questions that aren’t the same.

Three tiers run in order, each consuming markets from a shared pool.

Tier 0 exists because text matching under-performs on a categorical event — it only “sees” the candidates whose titles happen to look alike. Declaring the correspondence at the *event* level instead means the whole field pairs. Adding a league is one registry entry; teams align by token overlap, so no roster data is needed. An alias table keeps *Donald Trump Jr* distinct from *Donald Trump* — exactly the collision that would otherwise invent a huge spread.

Tier	Method	How	Confidence
0	Entity alignment	A registry maps a Polymarket event to a Kalshi series; every outcome aligns by canonical entity. 6 events, ~108 sports pairs.	VERIFIED
1	Curated seed	6 hand-verified pairs, carrying both a live and a snapshot id.	VERIFIED
2	IDF-weighted fuzzy	Weighted token overlap behind four guards, threshold 0.6.	FUZZY

Table 2. Confidence degrades down the stack. Only VERIFIED pairs can be presented as arbitrage.

Tier 2 scores weighted token overlap. With $T(m)$ the token set of market m after stopwords, stemming and alias expansion, over N documents:

$$\text{idf}(t) = \log\left(1 + \frac{N}{\text{df}(t)}\right), \quad S(p, k) = \frac{\sum_{t \in T(p) \cap T(k)} \text{idf}(t)}{\min(W(p), W(k))}, \quad W(m) = \sum_{t \in T(m)} \text{idf}(t). \quad (1)$$

Accept only if $S \geq 0.6$, at least 2 tokens are shared, and at least one shared token has $\text{df}(t) \leq 8$.

That last condition is what makes the tier usable. Every 2028 candidate shares 2028, president, nominee — without a rarity requirement they all match each other. Requiring a rare token forces the match to rest on a name.

Four guards reject a candidate before scoring. Two were added to kill specific bad matches found in testing:

- **Question kind** — *nominee / run / winner / host* never cross-match. This keeps “will X win the World Cup” apart from “will X host it”: same entity, near-identical tokens, very different prices.
- **Multi-entity** — a combined “A, B or C” market never matches a single-entity one.
- **Numbers** — conflicting numerals separate “ $\geq 25\text{bps}$ ” from “50bps”.
- **Category** — must be equal.

After scoring, one more check: a fuzzy pair with spread > 0.35 is dropped. Two equivalent liquid markets don’t disagree by 35 points, so that gap is almost certainly a title collision.

4 Arbitrage

Differencing mid-prices gives an edge you cannot transact. You buy at the ask and sell at the bid, and size at the touch is usually trivial.

On a binary market, YES on one venue plus NO on the other redeems for exactly \$1. Buying NO at $1 - b$ equals selling YES at bid b . So for a share bought at ask a and hedged at bid b :

$$\text{net edge/share} = b - a - \phi_{\text{buy}}(a) - \phi_{\text{sell}}(1 - b), \quad \text{capital/share} = a + (1 - b), \quad (2)$$

with $\phi_{\text{kalshi}}(p) = 0.07 p (1 - p)$ and $\phi_{\text{polymarket}}(p) = 0$. Note the shape: [Kalshi](#)’s fee peaks at $p = 0.5$, so a flat fee assumption would misprice exactly the mid-probability markets where spreads are widest.

The engine walks both ladders with two pointers — asks ascending against bids descending, consuming the smaller side each step, halting when the fee-adjusted edge goes non-positive. Both directions are tried; the better wins.

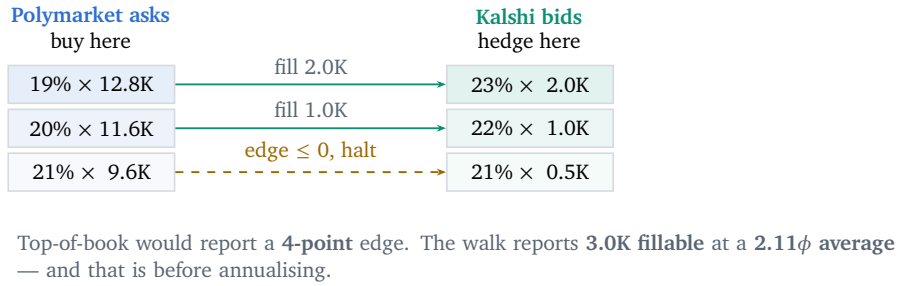


Figure 3. The book walk (illustrative sizes). The reported edge averages over filled size, not the touch.

4.1 Annualisation — the decisive step

A cross-exchange lock is not a free roll. Capital sits committed until resolution, and for the 2028 market that is over two years.

$$\text{ROI} = \frac{\text{net profit}}{\text{capital}}, \quad \text{ROI}_{\text{ann}} = \frac{\text{ROI}}{\tau}, \quad \tau = \frac{t_{\text{close}} - t_{\text{now}}}{365.25 \text{ days}}. \quad (3)$$

Reported only when $\tau > 0.02$ years, below which annualising produces nonsense multipliers. Benchmarked against a configurable risk-free rate (6%/yr default): green when it clears, amber when a Treasury bill beats it.

This is the most important number in the product. It turns the 8-point gap of \$1 into 0.8%/year — and shows it in amber.

Two spread notions, on purpose. `lib/match.ts` computes a naive $|p_{\text{poly}} - p_{\text{kalshi}}|$ with a flat 1% buffer: a cheap ranking pre-filter that runs corpus-wide and needs no order book. `lib/arb.ts` does the real walk: the decision number. So the radar ranks by the naive spread but reports the rigorous one — net profit scales with size deployed, making it a poor sort key.

5 Resolution Divergence

Price analysis cannot catch the failure that matters most: two markets that are the same question by every measure, but settle differently.

Kalshi resolves the 2028 presidential market on who is **inaugurated** on 20 January 2029. **Poly-market** resolves on who **wins**, as called by AP, Fox and NBC in November 2028. In the clean case they agree. They differ in source, threshold and timing.

Worst case, shown in the UI. Hold YES-**Polymarket** + NO-**Kalshi** on candidate X. The networks call X in November, so **Polymarket** resolves YES — that leg loses. Then X dies or is replaced before inauguration, someone else takes office, and **Kalshi** resolves NO — the hedge loses too. A “risk-free” position that loses both sides.

An LLM equivalence judge was run over each matched event’s resolution text and its output reviewed, classifying pairs equivalent / minor / divergent with concrete divergence cases. Results are baked in rather than computed per request — resolution criteria change on the order of years,

so a live call would be latency for no freshness. Unjudged pairs get a generic warning rather than silently appearing safe.

The judge isn't rubber-stamping: both nomination markets came back minor, not equivalent, because **Polymarket** locks the result against a post-convention replacement and **Kalshi** is silent on it.

6 Engineering Notes

Normalisation. One invariant: all prices are floats in $[0, 1]$, converted once inside each adapter. One guard worth noting — a market maker parking $1\phi/99\phi$ produces a meaningless “50%” midpoint, so any two-sided quote wider than 90ϕ defers to last trade or returns `null`. Better nothing than a confident wrong number.

Real-time. **Polymarket**'s CLOB publishes over WebSocket; the server holds the socket and bridges it to the browser over SSE. The client layers SSE over a 2s poll, so the first paint is instant and the stream takes over once connected. **Kalshi** needs API-key auth for its socket, so it polls — the venue's auth model, not an implementation gap.

Reachability. **Polymarket** is DNS-blocked on some networks (notably India). The block is DNS-only, so the app resolves over DoH and connects to the IP with correct SNI. Live data, no VPN. The chain degrades direct DNS $\rightarrow$ DoH $\rightarrow$ PolyRouter $\rightarrow$ stale cache $\rightarrow$ labeled snapshot. `mode: "snapshot"` propagates through the type system to every badge and renders history dashed, so demo data can never appear live.

Caching. The corpus build costs ~ 17 s cold. Corpus (180 s), pairs (15 s) and arb (20 s) caches are stale-while-revalidate with in-flight dedup, so only the first build blocks. A boot hook prewarms them every 150 s — inside the 180 s TTL.

Chart alignment. The charting library lays out points by *index*, not timestamp. Irregular points give a time axis whose labels don't match positions, and two venues' series that silently drift apart. Everything is resampled onto a shared uniform grid, so index is proportional to time and the venues align at every x .

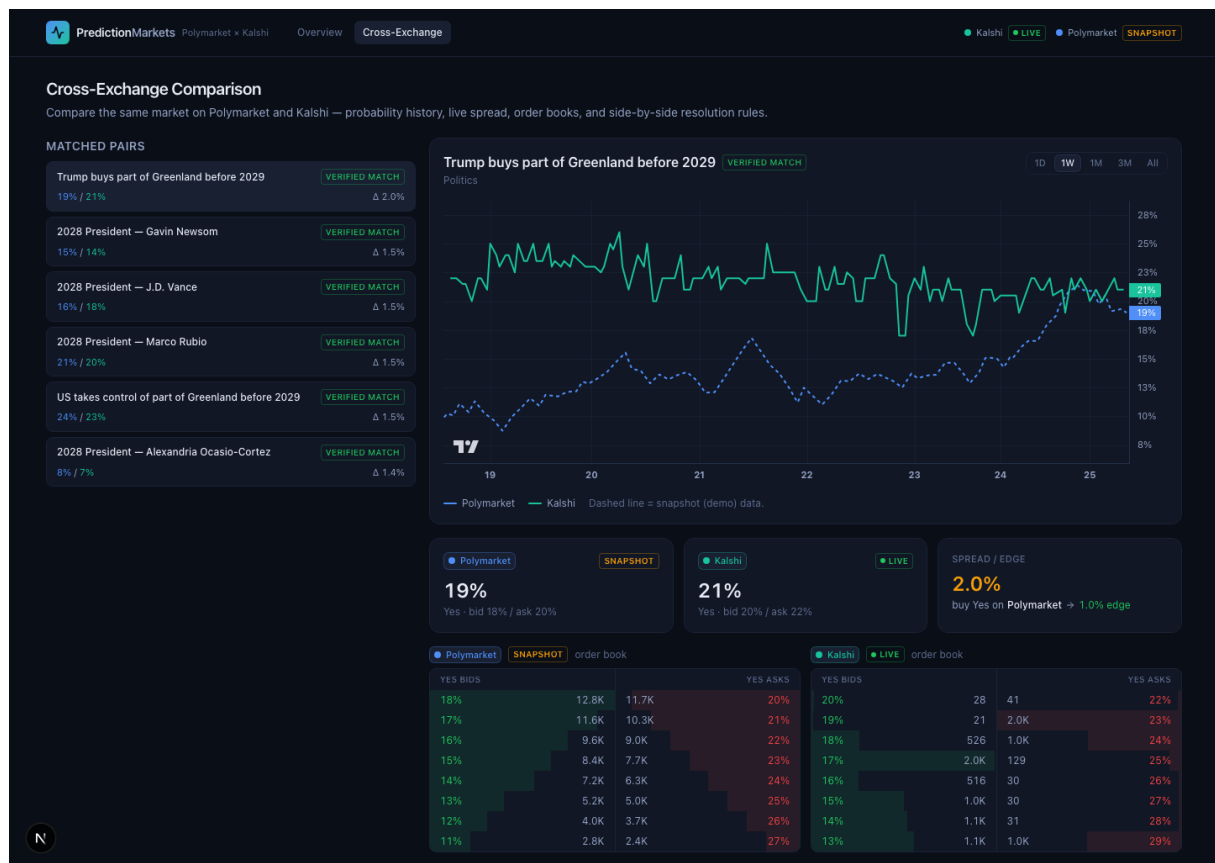


Figure 4. Cross-exchange comparison. Dashed line = snapshot, so provenance is visible in the mark itself. The spread tile reports the raw 2.0% gap and the 1.0% fee-adjusted edge.

7 Limitations

No automated tests. The most conspicuous gap. Best targets are pure and dependency-free: the fuzzy scorer and its guards, the arbitrage walk, the resampler. The walk especially — a two-pointer accumulator decrementing across two ladders is where an off-by-one goes silent and returns a plausible wrong number. First thing I’d add.

Fee model in three places. The flat 1% buffer appears in the matcher and the compare view; the non-linear curve lives in the arb engine. The split is deliberate, but the constant should be defined once.

Duplication. The SWR plus in-flight-dedup idiom is written four times with slightly different error handling. DoH resolution is implemented twice. Two components (`SpreadTable`, `Sparkline`) are dead, and the README still mentions a “spread table” that no longer exists.

Inherent constraints. Both venues cap history at one-minute granularity. Volume units aren’t comparable (**Polymarket** USD, **Kalshi** contract count) and are deliberately not converted. Only three events are individually judged for divergence. Execution is out of scope — the system sizes opportunities, it does not trade.

The most valuable output here is often negative: an amber ROI under the risk-free line, or a flag explaining how a lock loses both legs. A tool that only ever finds opportunities isn’t measuring anything.

Appendix

Self-contained reference: API surface, data model, configuration and algorithms — enough to run, extend or review the system without reading the source.

A HTTP API

All GET-only, Node runtime, no auth, no mutating endpoints. stale-while-revalidate is $6 \times s\text{-maxage}$.

Endpoint	Parameters	Returns	s-maxage
/api/markets	q, category, venue, sort, offset, limit	PagedMarketsResponse	30 s
/api/market/[venue]/[id]	path params	Market	15 s
/api/orderbook/[venue]/[id]	path params	Orderbook	1 s
/api/stream/orderbook/[venue]/[id]	400 for kalshi	text/event-stream	—
/api/history	range, repeated m	HistoryResponse	60 s
/api/quote	repeated m (max 12)	QuoteResponse	1 s
/api/pairs	—	PairsResponse	30 s
/api/arb	—	ArbResponse	10 s
/api/search	q	SearchResponse	60 s
/api/stats	—	StatsResponse	30 s
/api/status	—	VenueStatus[]	15 s

Table 3. /api/markets with q also fans out to [Polymarket](#)’s public search. The SSE stream emits data: frames carrying an Orderbook, plus a 15 s heartbeat.

Upstreams. Both venues’ read endpoints are public and need no key.

- **Kalshi** — /events?with_nested_markets (corpus), /markets/{t}/[orderbook], /series/{s}/markets/{t}/candlesticks
- **Polymarket** — Gamma /markets and /events?slug=, CLOB /book and /prices-history, plus the CLOB WebSocket

Kalshi returns fixed-point cents and volume as a contract count; **Polymarket** returns JSON-encoded *strings* for outcomes, and volume in USD. All requests omit Origin.

B Data Model and Configuration

Invariant: every price is a float in $[0, 1]$, converted once inside each venue adapter. Vendor wire shapes stay private to their adapter.

Type	Definition
Venue	"polymarket" "kalshi"
DataMode	"live" "snapshot" "unreachable" — drives every badge and the dashed-line treatment
Market	id = \${venue}:\${nativeId}; title, subtitle, category, mode; yesPrice, bid, ask; volumes; closeTime; rules (read by the divergence layer); tokenIds (Polymarket); seriesTicker (Kalshi)
PricePoint	{ t, p } — t in unix seconds
Orderbook	bids descending, asks ascending, of { price, size }
MarketPair	both markets , confidence (verified fuzzy unmatched), matchMethod, spread, edge, cheaperVenue
Divergence	status (equivalent minor divergent), confidence, divergenceCases[], worstCase
ArbResult	buyVenue, sellVenue, fillableSize, netProfit, capital, roi, annualizedRoi, avgEdge, bestEdge

Variable	Default	Effect
POLY_DOH	auto	auto = direct then DoH on failure; on; off
DOH_RESOLVER	Cloudflare	DoH endpoint
POLYROUTER_API_KEY	unset	Enables the PolyRouter stage
KALSHI_BASE_URL	Kalshi v2	Override venue base
POLY_GAMMA_URL	/	Override host
POLY_CLOB_URL	Polymarket	
	hosts	
POLY_SNAPSHOT	on	off disables the demo fallback
KALSHI_PAGES	/ 4 / 4	Breadth vs cold-start latency
POLY_PAGES		
UPSTREAM_TIMEOUT_MS	12000	Upstream timeout
NEXT_PUBLIC_RISK_FREE_RATE	0.06	Annualised-ROI benchmark

Table 4. All optional — both venues work with no key. Copy `.env.example` to `.env.local`. Registered events: KXPRESPERSON, KXPRESNOMD, KXPRESNOMR, KXMENWORLD CUP, KXNBA, KXNHL.

C Algorithms

```

WALK(buyAsks, buyVenue, sellBids, sellVenue):
  asks <- buyAsks sorted ASCENDING ; bids <- sellBids sorted DESCENDING
  i, j <- 0, 0 ; remainingAsk <- asks[0].size ; remainingBid <- bids[0].size

  WHILE i < |asks| AND j < |bids|:
    a <- asks[i].price ; b <- bids[j].price
    IF b <= a: BREAK // no gross edge left
    netPer <- b - a - FEE(buyVenue, a) - FEE(sellVenue, 1 - b)
    IF netPer <= 0: BREAK // fees consumed the edge
    qty <- MIN(remainingAsk, remainingBid)
    IF qty <= 0: BREAK

    IF size == 0: bestEdge <- netPer // edge at the touch
    size += qty ; net += netPer * qty
    capital += (a + (1 - b)) * qty // YES leg + NO leg
    remainingAsk -= qty ; remainingBid -= qty
    IF remainingAsk == 0: remainingAsk <- asks[++i].size

```

```

    IF remainingBid ~= 0: remainingBid <- bids[++j].size
    RETURN { size, net, capital, bestEdge }

```

```

FEE(venue, p): RETURN 0.07 * p * (1 - p) IF venue == kalshi ELSE 0

```

```

COMPUTE-ARB(polyBook, kalshiBook, closeTime):
    best <- better of WALK(poly->kalshi) and WALK(kalshi->poly)
    IF best.net <= 0: RETURN EMPTY_ARB
    roi <- best.net / best.capital
    years <- (closeTime - now) / 365.25 days
    RETURN { roi, annualizedRoi: roi/years IF years > 0.02 ELSE null,
            avgEdge: best.net / best.size }

```

```

BUILD-PAIRS(kalshiMarkets, polyMarkets, eventInputs):
    // Tier 0 -- event-level entity alignment -> VERIFIED
    FOR EACH (eventMatch, eventPolyMarkets) IN eventInputs:
        kByEntity <- MAP canonicalEntity(m.subtitle) -> m, over the Kalshi series
        FOR EACH p IN eventPolyMarkets NOT IN usedP:
            k <- kByEntity[canonicalEntity(p.subtitle)]
            IF not found AND matchBy == "team":
                k <- best token-overlap match among unused, requiring overlap >= 1
            IF k found AND k NOT IN usedK: EMIT pair(p, k, VERIFIED)

    // Tier 1 -- curated seed -> VERIFIED
    FOR EACH s IN SEED_PAIRS not already consumed: EMIT pair(..., VERIFIED)

    // Tier 2 -- IDF-weighted fuzzy -> FUZZY
    df <- document frequency over the remaining markets on both venues
    idf <- t -> log(1 + N / df[t])
    index <- inverted index: token -> [kalshi indices] // blocking

    FOR EACH p IN the 600 most liquid remaining Polymarket markets:
        candidates <- kalshi markets sharing >= 2 tokens, top 40
        FOR EACH k IN candidates:
            IF p.category != k.category: CONTINUE // guard 1
            IF numbersConflict(p, k): CONTINUE // guard 2
            IF questionConflict(p, k): CONTINUE // guard 3
            IF isMultiEntity(p) OR isMultiEntity(k): CONTINUE // guard 4
            TRACK best WEIGHTED-SCORE(tokens(p), tokens(k)) >= 0.60
            IF best found AND pair.spread <= 0.35: EMIT pair(p, k, FUZZY)
            // else: title collision

    SORT BY (verified, fuzzy, unmatched), THEN spread DESCENDING

WEIGHTED-SCORE(a, b):
    shared <- a INTERSECT b
    IF |shared| < 2: RETURN 0
    IF NO t IN shared WITH df[t] <= 8: RETURN 0 // needs a distinctive token
    RETURN SUM(idf[t] for t in shared) / MIN(weight(a), weight(b))

```

Series resampling. Pick a per-range bucket width, cap the grid at 2,200 buckets, forward-fill: for each bucket time g , emit the most recent PricePoint with $t \leq g$. Every series shares the same grid, so index is proportional to time.

D Build and Run

```

npm install
npm run dev # http://localhost:3000
npm run build && npm run start # production

```

No configuration needed. **Kalshi** is live out of the box; **Polymarket** works directly where DNS resolves it and over DoH where it doesn't. Expect ~17 s of cold build at boot, absorbed before the first request.

To test degradation: point POLY_GAMMA_URL at an unroutable host. The badge should flip to

SNAPSHOT, the banner appear, [Polymarket](#) history render dashed, and every view keep working.

To rebuild this report: tectonic report.tex, or pdflatex report.tex twice.